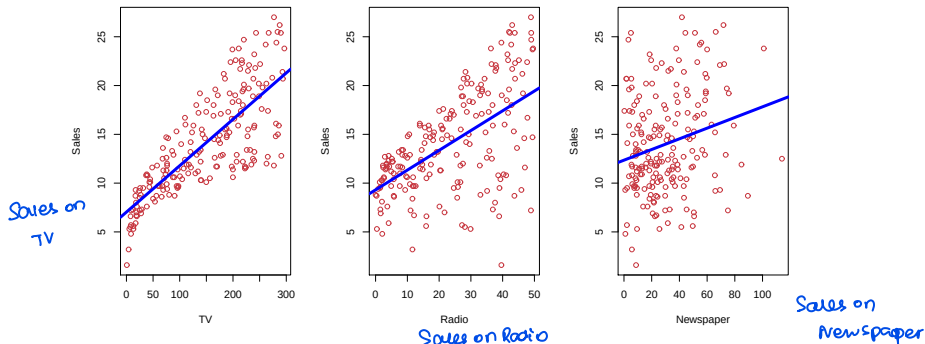


What is Statistical Learning?



Shown are **Sales** vs **TV**, **Radio** and **Newspaper**, with a blue linear-regression line fit separately to each.

Can we predict **Sales** using these three?

Perhaps we can do better using a model

$$\text{Sales} \approx f(\text{TV}, \text{Radio}, \text{Newspaper})$$

Response variables.

Sales → explanatory var

Notation

Here **Sales** is a *response* or *target* that we wish to predict. We generically refer to the response as Y .

TV is a *feature*, or *input*, or *predictor*; we name it X_1 .

Likewise name **Radio** as X_2 , and so on.

We can refer to the *input vector* collectively as

$$\underline{X} = \begin{pmatrix} X_1 \\ X_2 \\ \underline{X_3} \end{pmatrix} \begin{matrix} \rightarrow \text{TV} \\ \rightarrow \text{Newspaper} \\ \rightarrow \text{Radio} \end{matrix}$$

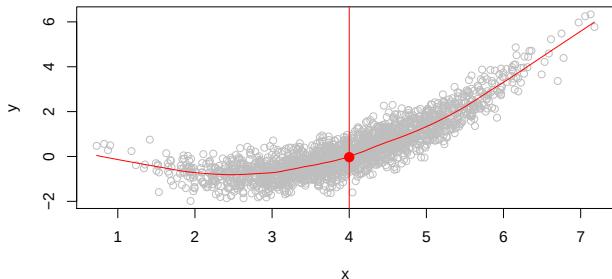
Now we write our model as

$$\text{vector?} \quad \leftarrow Y = f(X) + \textcircled{\epsilon} \text{ Irreducible error -}$$

where ϵ captures measurement errors and other discrepancies.

What is $f(X)$ good for?

- With a good f we can make predictions of Y at new points $X = x$.
- We can understand which components of $X = (X_1, X_2, \dots, X_p)$ are important in explaining Y , and which are irrelevant. e.g. **Seniority** and **Years of Education** have a big impact on **Income**, but **Marital Status** typically does not.
- Depending on the complexity of f , we may be able to understand how each component X_j of X affects Y .



Is there an ideal $f(X)$? In particular, what is a good value for $f(X)$ at any selected value of X , say $X = 4$? There can be many Y values at $X = 4$. A good value is

$$f(4) = E(Y|X = 4)$$

→ 2000 bcz can help reduce MSE.

$E(Y|X = 4)$ means *expected value* (average) of Y given $X = 4$.

This ideal $f(x) = E(Y|X = x)$ is called the *regression function*.

The regression function $f(x)$

Reducible
errors
we consider

$$= (f(x) - \hat{f}(x))^2$$

Is also defined for vector X ; e.g. $\hookrightarrow$ Ideal $f(x)$ that allows you to

$f(x) = f(x_1, x_2, x_3) = E(Y|X_1 = x_1, X_2 = x_2, X_3 = x_3)$ estimate the value of $f(x)$ for a given x .

- Is the *ideal* or *optimal* predictor of Y with regard to mean-squared prediction error: $f(x) = E(Y|X = x)$ is the function that minimizes $E[(Y - g(X))^2|X = x]$ over all functions g at all points $X = x$.
- $\epsilon = Y - f(x)$ is the *irreducible* error — i.e. even if we knew $f(x)$, we would still make errors in prediction, since at each $X = x$ there is typically a distribution of possible Y values.
- For any estimate $\hat{f}(x)$ of $f(x)$, we have

$$E[(Y - \hat{f}(X))^2|X = x] = \underbrace{[f(x) - \hat{f}(x)]^2}_{\text{Reducible}} + \underbrace{\text{Var}(\epsilon)}_{\text{Irreducible}}$$

$\nearrow$ Irreducible due to variance

How to estimate f

- Typically we have few if any data points with $X = 4$ exactly.
- So we cannot compute $E(Y|X = x)$! *→ we don't have data points exactly*
- Relax the definition and let

$$\hat{f}(x) = \text{Ave}(Y|X \in \mathcal{N}(x))$$

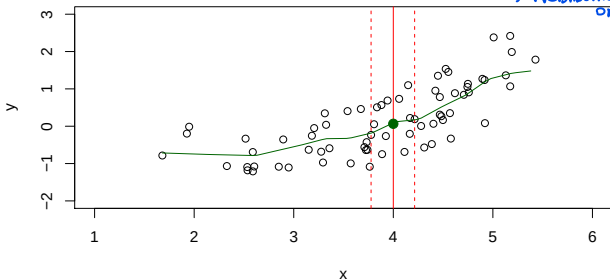
where $\mathcal{N}(x)$ is some *neighborhood* of x .

↓
∴ we consider the neighborhood

→ Take a min margin for what
to be considered as
a neighborhood

$$\therefore f(x) = E(Y|X=x) \rightarrow E(Y|X \sim \mathcal{N}(x))$$

↪ Neighborhood of x



$p \rightarrow \# \text{ predictors}$.

- Nearest neighbor averaging can be pretty good for small p — i.e. $p \leq 4$ and large-ish N . (So more NN's can be present (i.e. closer to data point))
- We will discuss smoother versions, such as kernel and spline smoothing later in the course.
- Nearest neighbor methods can be *lousy* when p is large. Reason: the curse of dimensionality. Nearest neighbors tend to be far away in high dimensions.

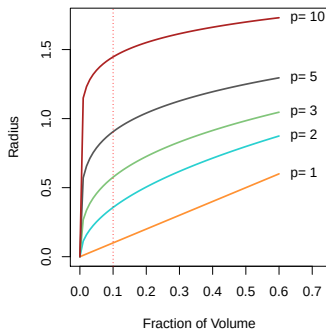
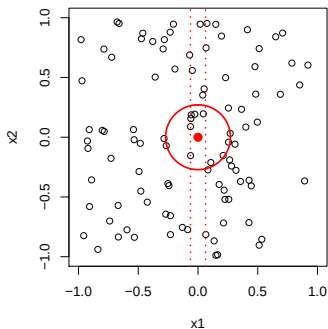
$p(1-h)$
 $\downarrow$
NN are closer to our data point.

With d with large dimension, nearest neighbors tend to be far away.

- We need to get a reasonable fraction of the N values of y_i to average to bring the variance down—e.g. 10%.
- A 10% neighborhood in high dimensions need no longer be local, so we lose the spirit of estimating $E(Y|X = x)$ by local averaging.

The curse of dimensionality

10% Neighborhood



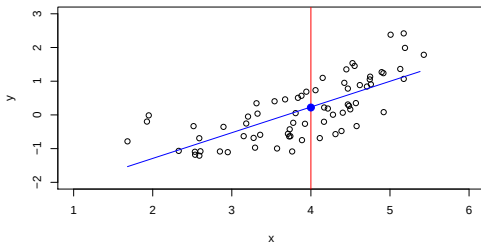
Parametric and structured models

The *linear* model is an important example of a parametric model:

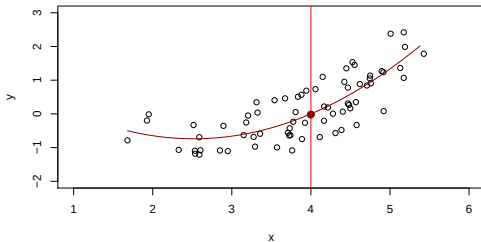
$$f_L(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots \beta_p X_p.$$

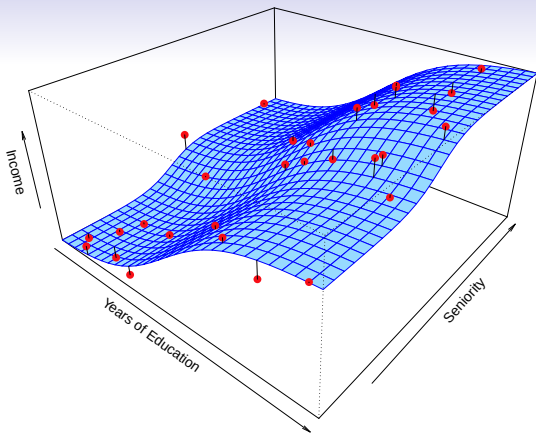
- A linear model is specified in terms of $p + 1$ parameters $\beta_0, \beta_1, \dots, \beta_p$.
- We estimate the parameters by fitting the model to training data.
- Although it is *almost never correct*, a linear model often serves as a good and interpretable approximation to the unknown true function $f(X)$.

A linear model $\hat{f}_L(X) = \hat{\beta}_0 + \hat{\beta}_1 X$ gives a reasonable fit here



A quadratic model $\hat{f}_Q(X) = \hat{\beta}_0 + \hat{\beta}_1 X + \hat{\beta}_2 X^2$ fits slightly better.

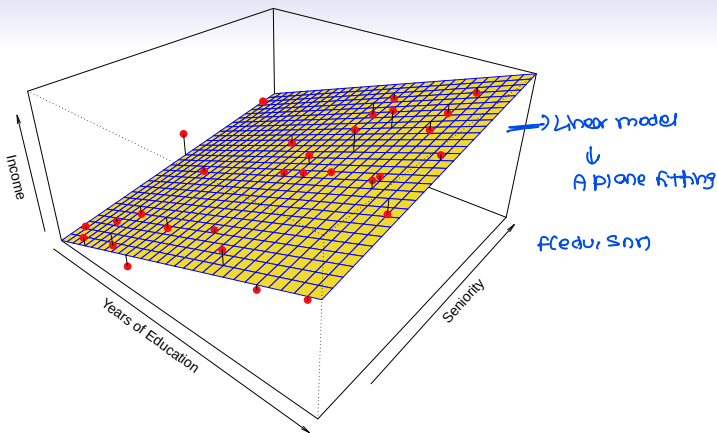




Simulated example. Red points are simulated values for **income** from the model

$$\text{income} = f(\text{education}, \text{seniority}) + \epsilon$$

f is the blue surface.



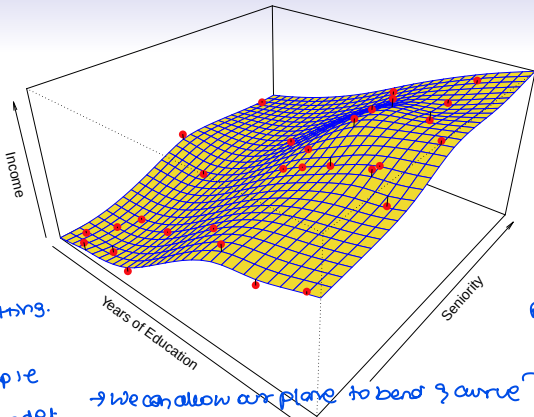
Linear regression model fit to the simulated data.

$$\hat{f}_L(\text{education}, \text{seniority}) = \hat{\beta}_0 + \hat{\beta}_1 \times \text{education} + \hat{\beta}_2 \times \text{seniority}$$

$$f(x_1, x_2) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 \quad | \quad w_0 x + \text{bias} \quad w \rightarrow \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} \quad x \rightarrow \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} \quad w \cdot x + w_0 x$$

$w_1 x_1 + w_2 x_2$

Thin plate
Spline $\rightarrow$ 2/more
dimensions...



We can also go to
some complex
repr for data fitting.

$\downarrow$
Not just simple
Linear model

$\rightarrow$ We can allow our plane to bend & curve
 $\rightarrow$ Follow & fit data more clearly

Bias &
Variance
Tradeoff.
 $\uparrow$

When we want to
fit data more perfectly
our repr becomes
complex.

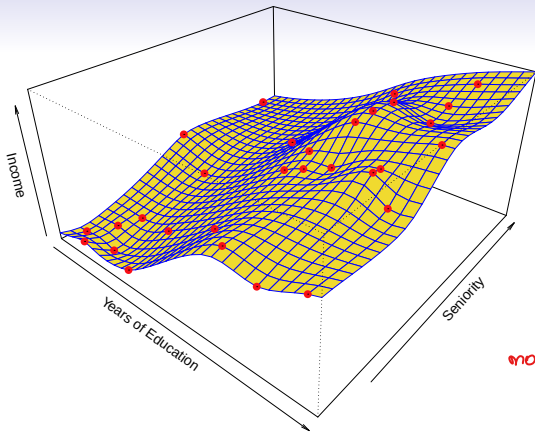
More flexible regression model $\hat{f}_S(\text{education}, \text{seniority})$ fit to
the simulated data. Here we use a technique called a thin-plate
spline to fit a flexible surface. We control the roughness of the
fit (chapter 7).

$\rightarrow$ Flexible curve used to fit data.
Spline $\rightarrow$ Piecewise polynomial fit

$\rightarrow$ primarily 1 dimensional (line)

too smooth $\rightarrow$ underfitting

too rough $\rightarrow$ overfitting
(Sharp)

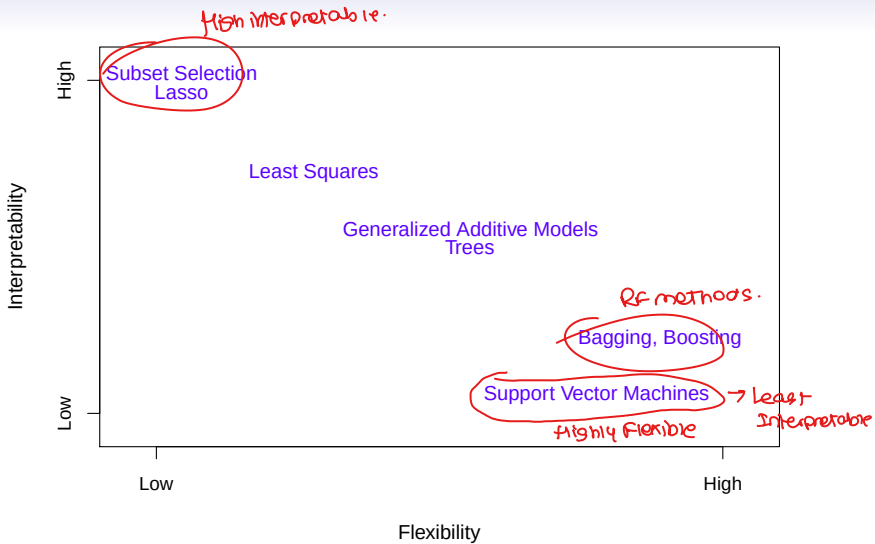


more flexible
↳ overfitting.
A

Even more flexible spline regression model $\hat{f}_S(\text{education}, \text{seniority})$ fit to the simulated data. Here the fitted model makes no errors on the training data! Also known as *overfitting*.

Some trade-offs

- ✓ Prediction accuracy versus interpretability.
 - Linear models are easy to interpret; thin-plate splines are not.
- Good fit ✓ versus over-fit or under-fit.
 - How do we know when the fit is just right?
- Parsimony versus black-box. ✓
 - We often prefer a simpler model involving fewer variables over a black-box predictor involving them all.



Assessing Model Accuracy

Suppose we fit a model $\hat{f}(x)$ to some training data $\text{Tr} = \{x_i, y_i\}_1^N$, and we wish to see how well it performs.

- We could compute the average squared prediction error over Tr :

$$\text{MSE}_{\text{Tr}} = \text{Ave}_{i \in \text{Tr}} [y_i - \hat{f}(x_i)]^2$$

Overfit models can understand training data very well

↓

This may be biased toward more overfit models.

∴ Training error is not a

- Instead we should, if possible, compute it using fresh test data $\text{Te} = \{x_i, y_i\}_1^M$:

Good measure

↓
unseen.

to see how

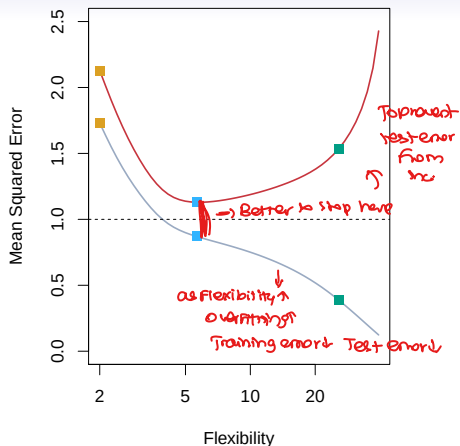
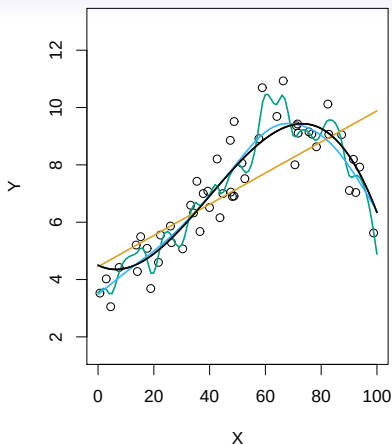
well model performs

$$\text{MSE}_{\text{Te}} = \text{Ave}_{i \in \text{Te}} [y_i - \hat{f}(x_i)]^2$$

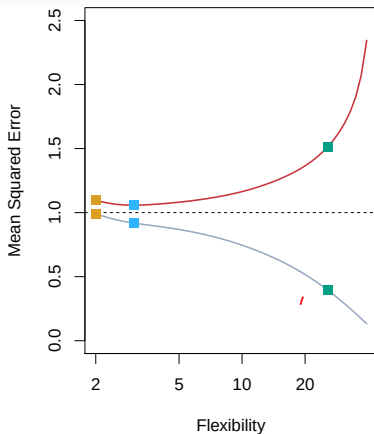
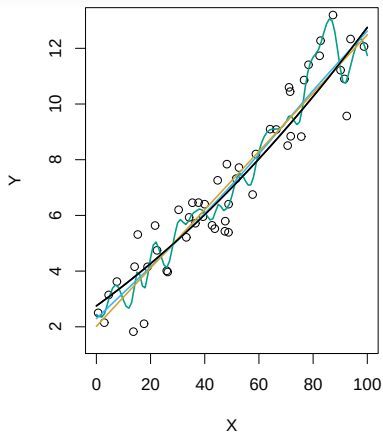
↑
MSE

↓
actual
value

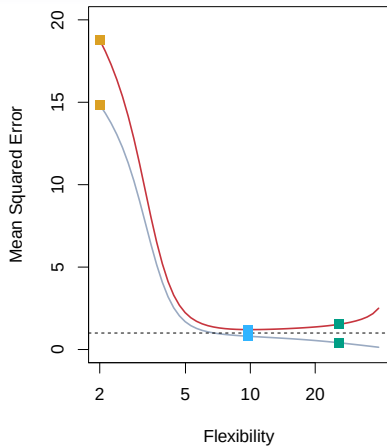
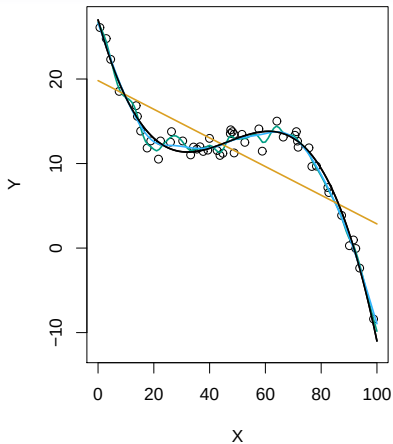
↓
predicted
value



Black curve is truth. Red curve on right is MSE_{Te} , grey curve is MSE_{Tr} . Orange, blue and green curves/squares correspond to fits of different flexibility.



Here the truth is smoother, so the smoother fit and linear model do really well.



Here the truth is wiggly and the noise is low, so the more flexible fits do the best.

Bias-Variance Trade-off

Suppose we have fit a model $\hat{f}(x)$ to some training data Tr , and let (x_0, y_0) be a test observation drawn from the population. If the true model is $Y = f(X) + \epsilon$ (with $f(x) = E(Y|X = x)$), then

$$E \left(y_0 - \hat{f}(x_0) \right)^2 = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon).$$

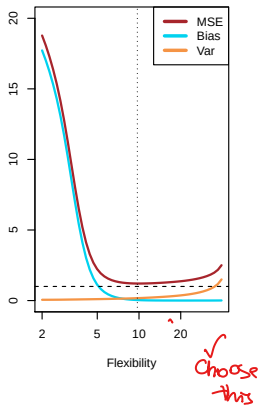
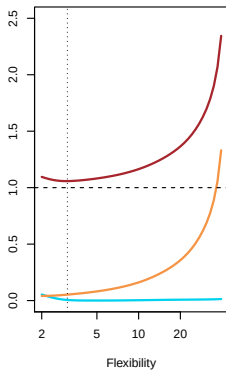
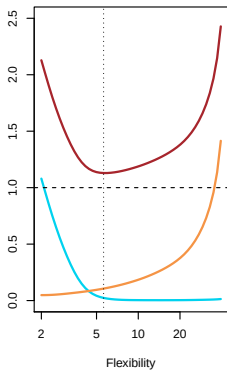
The expectation averages over the variability of y_0 as well as the variability in Tr . Note that $\text{Bias}(\hat{f}(x_0)) = E[\hat{f}(x_0)] - f(x_0)$.

Handwritten notes: "Diff by actual value?" with an arrow pointing to $f(x_0)$; "Predicted value" with an arrow pointing to $E[\hat{f}(x_0)]$.

Typically as the *flexibility* of $\hat{f}$ increases, its variance increases, and its bias decreases. So choosing the flexibility based on average test error amounts to a *bias-variance trade-off*.

Handwritten note: $\therefore$ As flexibility (how well it can fit) $\uparrow$ variance $\uparrow$, bias $\downarrow$

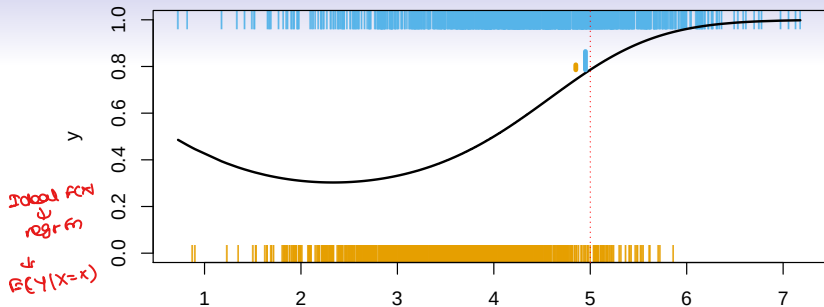
Bias-variance trade-off for the three examples



Classification Problems

Here the response variable Y is *qualitative* — e.g. email is one of $\mathcal{C} = (\text{spam}, \text{ham})$ (ham =good email), digit class is one of $\mathcal{C} = \{0, 1, \dots, 9\}$. Our goals are to:

- Build a classifier $C(X)$ that assigns a class label from $\mathcal{C}$ to a future unlabeled observation X .
- Assess the uncertainty in each classification
- Understand the roles of the different predictors among $X = (X_1, X_2, \dots, X_p)$.



?? assigns a class label^x to data point^x

Is there an ideal $C(X)$? Suppose the K elements in $\mathcal{C}$ are numbered $1, 2, \dots, K$. Let

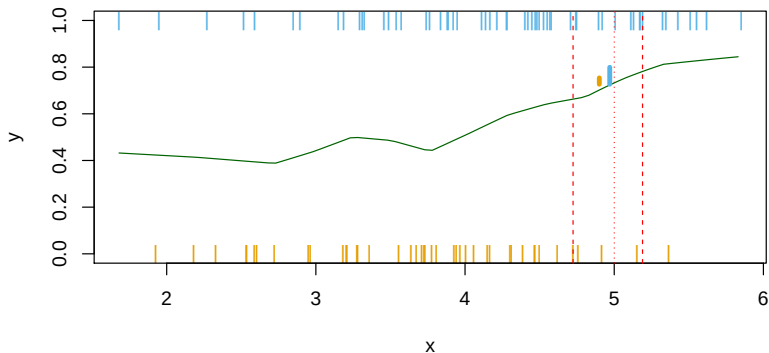
$$p_k(\underline{x}) = \Pr(\underline{Y} = \underline{k} | \underline{X} = \underline{x}), \quad k = 1, 2, \dots, K.$$

These are the *conditional class probabilities* at x ; e.g. see little barplot at $x = 5$. Then the *Bayes optimal* classifier at x is

Handwritten red notes on the right of the text:

- See which class has major prob at the datapoint?
- max: assign that label to the datapoint

$$C(x) = j \text{ if } p_j(x) = \max\{p_1(x), p_2(x), \dots, p_K(x)\} \rightarrow \text{Ideal classifier}$$



Nearest-neighbor averaging can be used as before.
Also breaks down as dimension grows. However, the impact on $\hat{C}(x)$ is less than on $\hat{p}_k(x)$, $k = 1, \dots, K$.

Classification: some details

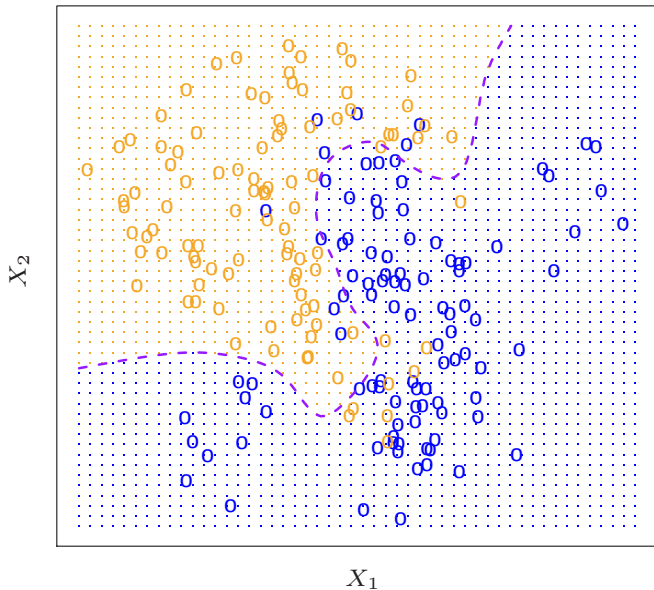
- Typically we measure the performance of $\hat{C}(x)$ using the misclassification error rate:

$$\text{Err}_{\text{Te}} = \text{Ave}_{i \in \text{Te}} I[y_i \neq \hat{C}(x_i)]$$

In reg $\frac{1}{n} \sum_{i \in \text{Te}} (\hat{y}_i - \hat{f}(x_i))^2$

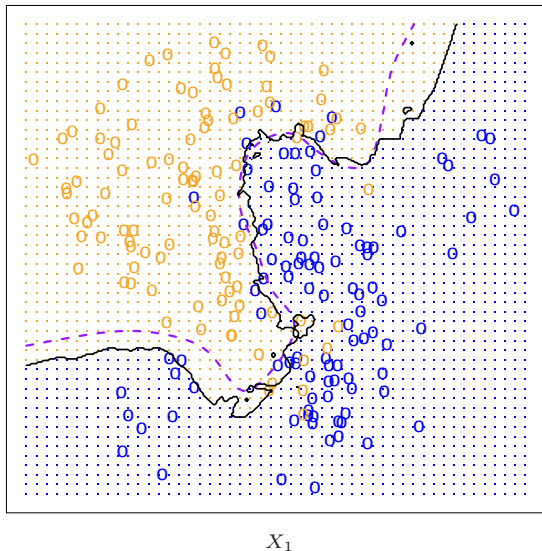
- The Bayes classifier (using the true $p_k(x)$) has smallest error (in the population). → The ideal C(x)
- Support-vector machines build structured models for $C(x)$.
- We will also build structured models for representing the $p_k(x)$. e.g. Logistic regression, generalized additive models.

Example: K-nearest neighbors in two dimensions

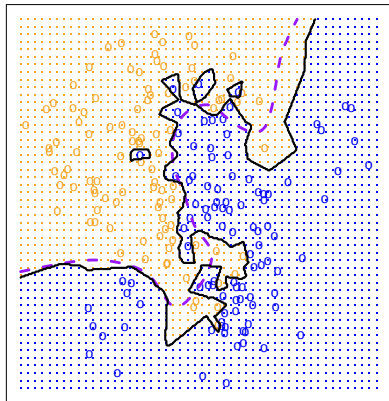


KNN: K=10

$\therefore$ classifier
 $\downarrow$
Several options
KNN logreg
SVM Tree based
methods.



KNN: $K=1$

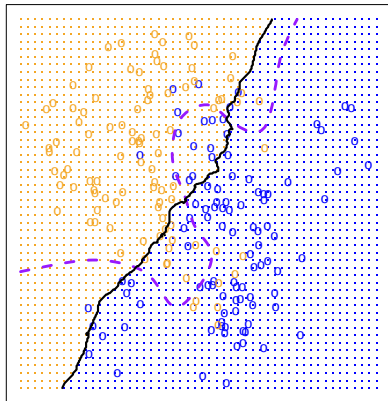


Overfitting

$K=1 \rightarrow$ very flexible $\rightarrow$ fits
training data
very well

But has high variance

KNN: $K=100$



Longer $K \rightarrow$ noise is added in
pred

$\therefore$ Training $\uparrow$
Test error $\downarrow$

